

Orbit Academy — Instructor's Guide

v5.5

Contents

Orbit Academy — Instructor's Guide	1
1. What this is	1
2. Download & install	2
3. Ways to run it	3
4. The data file (server mode)	6
5. How a student uses it	6
6. Teaching notes, module by module	6
7. Instructor dashboard (server mode)	15
8. Updating & testing	16
9. Troubleshooting	17
11. Deep dive — the complexities of L1 (halo) orbits	17
12. Credits & license	18

Orbit Academy — Instructor's Guide

A complete guide to running, hosting, and teaching with Orbit Academy. Written for an instructor or course administrator; no prior web-development experience assumed.

1. What this is

Orbit Academy is a self-contained, browser-based course that teaches orbital dynamics to non-specialists (built for JASON / US Space Force technical-staff training). It has three parts:

- **Simulators** (tutN.html) — live, interactive 3-D physics visualizations, one per module.
- **Worksheets** (worksheetN.html) — guided lessons that open beside each simulator: learning objectives, a one-page tutorial, numbered hands-on exercises with check questions, a final quiz, a key-points summary, and further-reading links.
- **A course-management backend** (server.js) — optional. Provides student accounts, saved progress, prerequisite gating (you must finish Module N before N+1 unlocks), and an instructor roster dashboard.

The course covers **eight modules**: How Orbits Work → Angular Rates & Geosync → Naming Orbits & TLEs → Maneuvers & Perturbations → xGEO / Cislunar Space & Ref-

erence Frames → Lagrange Points & Complex Orbits → **Observability** → **Lunar Transfers & Artemis** (the piloting capstone). All eight modules are complete. (File-id note: Observability is Module 7 = tut7/worksheet7; the flight-sim capstone is Module 8 = tut8/worksheet8.) There is also a **course-wide final quiz** and a printable **completion certificate**.

2. Download & install

Everything is plain HTML/JavaScript plus one small Node.js server file. There is **nothing to compile and no packages to install** — ever. The one third-party component (the Three.js 3-D library) is held **inside the course folder**, so a running install makes no internet requests at all. Fetch it once with `bash tools/fetch-vendor.sh` (see §2.3); after that the course is fully self-contained and suitable for air-gapped machines.

2.1 Get the files

Option A — download a ZIP (no tools or GitHub knowledge needed): 1. In a web browser, go to <https://github.com/JASONSS26/orbit-academy> (sign in to GitHub if it asks — the project is private, so use the account that was invited to it). 2. Click the green <> **Code** button near the top-right of the file listing, then click **Download ZIP** in the menu that appears. The file `orbit-academy-main.zip` lands in your Downloads folder. 3. Unzip it — Windows: right-click → **Extract All...**; macOS: double-click. Inside the resulting folder is an **academy** folder: that's the whole course. Move it anywhere you like.

Option B — clone with git (if you have git):

```
git clone https://github.com/JASONSS26/orbit-academy.git
cd orbit-academy/academy
```

2.2 Confirm the assets are intact (optional, no internet)

There is nothing to download. Three.js and all four planet maps are committed to the repository, so the clone you just made is already a complete, runnable, offline course. This is deliberate: it removes the install step, and it removes version drift — every classroom runs identical bytes.

After a ZIP round-trip or a USB copy it is still worth confirming nothing was lost:

```
cd path/to/academy
bash tools/fetch-vendor.sh --check
```

It recomputes the SHA-384 of every vendored file and compares it with the table in `public/vendor/NOTICE.md`, which also records where each file came from and under what license. Expected tail: `PASS` — every vendored asset matches `public/vendor/NOTICE.md`. On Windows use Git Bash or WSL (`README.md` has a pure-PowerShell equivalent). See `docs/MAINTENANCE.md` §2.5 for the full asset architecture.

If it reports `MISSING` or `MISMATCH`, restore with `git checkout -- public/vendor` or re-copy the folder. A missing library also produces an explicit on-screen banner rather than a blank canvas.

2.3 Install Node.js (only needed to run the tracked/server mode)

What is Node.js and why do I need it? The course’s account/roster server (`server.js`) is a JavaScript program, and Node.js is the (free, ~50 MB) program that runs it — the same way you’d need Python installed to run a `.py` file. Installing it changes nothing else on the computer. You do **not** need it at all for the standalone (no-login) mode — see §3, Options 0 and 1.

Windows: 1. Go to <https://nodejs.org> and download the green **LTS** installer (a `.msi` file). 2. Run it and click through (Next → Next → Install). All defaults are fine; you do **not** need the “tools for native modules” checkbox. 3. Verify: open a **new** Command Prompt (press the Windows key, type `cmd`, press Enter) and type `node --version`. If it prints a version like `v22.x`, you’re set. - If it says ‘node’ is not recognized, the window was opened before the install finished — close it and open a fresh one (the PATH only updates for newly opened windows).

macOS: 1. Go to <https://nodejs.org> and download the **LTS** installer (a `.pkg` file), then run it. (Homebrew users: `brew install node` works too.) 2. Verify: open Terminal (⌘-space, type “Terminal”, Enter) and type `node --version`.

Any recent version works — the server uses only Node’s built-ins, nothing to `npm install`, ever.

Does this run on Windows? Yes — the whole course (server included) is plain Node.js with no OS-specific code, and runs identically on Windows, macOS, and Linux. The one exception is the *developer* test suite (`test/run.sh`), which is a bash script — on Windows run it from Git Bash or WSL. Instructors and students never need it.

3. Ways to run it

Choose based on whether you need **central accounts and a roster** — and note **Option 0** if the machines have no internet (the course is designed for that case and is verified against it).

How the pieces fit together

[Diagram — see the online version of this document.]

Use this to decide what you need before installing anything: the left-hand path needs nothing at all, the right-hand path needs Node.js and gives you the tools row.

Option 0 — Standalone / air-gapped (no internet, ever)

This is the default posture, and the one to use on a closed network. Orbit Academy makes **zero outbound network calls** when it runs: the 3-D library is loaded from inside the course folder, the planet maps resolve locally, and nothing phones home. There is no `npm install`, no build step, and no telemetry of any kind.

There is no preparation step. Every byte the course needs — the 3-D library included — is committed to the repository. Move the whole academy folder to the target machine:

copy it, zip it, clone it, or put it on a USB stick. Nothing has to be fetched first, on either end, and the target machine can be one that has never touched a network. On the target:

- **No-login use:** open `academy/public/gallery.html` in a browser. Done. (Start at *gallery*, not `index.html` — the hub is the server-mode page and needs `server.js`; it will say so and redirect you. Note also that from a `file://` URL Module 8’s forward window shows flat-shaded planets, since browsers block pixel reads on local images; it states this on screen and nothing else changes.)
- **Tracked cohort:** `node server.js`, then browse to `http://localhost:8080` (see Option 2).

Verify the posture at any time — both commands are safe to run on the closed network:

```
bash tools/fetch-vendor.sh --check      # what is present; whether anything could call out
node test/no-external-calls.test.js     # PROVES no outbound calls (static scan + live m
```

The second one is the important one for an accreditation conversation: it scans every shipped file for anything a browser would fetch, *and* separately executes all eight simulators against instrumented network primitives (`fetch`, `XMLHttpRequest`, `WebSocket`, `EventSource`, `sendBeacon`, `image` and `script src`) and fails if a single absolute URL is attempted. It should print `PASSED` — the course makes no outbound network calls. Air-gap clean.

Two honest caveats, neither of which sends anything anywhere:

- The worksheets carry ~99 **further-reading links** (Wikipedia, NASA, CelesTrak). They are ordinary links: nothing is fetched unless a student clicks one, and on a closed network they simply fail to open. They are listed in each worksheet’s *Further reading* panel if you would rather point students at an internal mirror.
- The Earth and Moon maps fall back to **schematic** versions that ship in the repo (ocean blue with a 15-degree graticule, marked equator and tropics). If `fetch-vendor.sh` also managed to pull the photographic maps, those are used instead. Both are local; the difference is cosmetic, and the schematic grid is arguably better for teaching rotation and inclination.

Option 1 — Standalone (zero install, no accounts)

The worksheets and simulators are static files. If no backend is present, each worksheet automatically runs in **standalone mode**: it renders normally and saves the student’s progress in that browser’s own storage (`localStorage`). No logins, no central roster — the course just works from a local file or any static host.

- **Run it with no server at all:** open `academy/public/gallery.html` directly in a browser, or serve the folder with any static server (e.g. `python3 -m http.server` from inside `academy/public/`).
- **Trade-off:** progress lives in each browser, so it doesn’t follow a student between devices, and there’s no roster. Best for demos, self-study, and quick reviewer previews.

A small badge (“● standalone — saved in this browser”) appears on the worksheet in this mode, so students know their progress is local.

□ **A note on GitHub Pages.** It’s tempting to host the static site on GitHub Pages, but on standard GitHub a **Pages site is readable by anyone with the**

URL even when the repo is private — there is no access control. Access-restricted Pages exists only on GitHub Enterprise Cloud. So do **not** use Pages if the material must stay limited to authorized people. For an authorized-only preview, have reviewers **clone the repo and run it locally** (they already have repo access), or use **Codespaces** (private port-forwarding), or a host behind your own SSO.

Option 2 — Full server (accounts, saved progress, instructor roster)

Run the Node server for a managed cohort with central accounts and a dashboard.

Easiest — no terminal needed: in the academy folder, double-click the launcher: - **Windows:** start-academy.bat - **macOS:** start-academy.command (first time, macOS may balk: right-click → **Open** → **Open**)

The launcher checks that Node.js is installed (and sends you to nodejs.org if it isn't), starts the server, and opens your browser to it. **Keep its window open** — closing it stops the server.

Or from a terminal:

```
cd path/to/academy          # tip: type "cd " then DRAG the academy folder onto the window
node server.js              # serves http://localhost:8080
```

Optional settings (note the syntax differs by shell):

macOS / Linux:

```
PORT=9000 node server.js
```

```
ORBIT_DATA=/path/to/academy_data.json node server.js
```

Windows Command Prompt:

```
set PORT=9000 && node server.js
```

Windows PowerShell:

```
$env:PORT=9000; node server.js
```

- Open **http://localhost:8080**. **The first account you register becomes the instructor.** Register yours first, then have students register their own.
- Progress and prerequisite gating are enforced server-side; the instructor account sees a roster of all students and their module/pass status.
- For a classroom, run it on one machine on the LAN and give students that machine's address (e.g. http://10.0.0.5:8080). For internet-facing use, put it behind HTTPS + a reverse proxy with rate-limiting (see docs/SECURITY.md).

Option 3 — Both

Publish the standalone build on GitHub Pages for broad/self-study access, **and** run the server for tracked cohorts. They share the same files.

	Standalone (Pages)	Full server
Install	none	Node.js
Accounts / login	no	yes (first user = instructor)

	Standalone (Pages)	Full server
Progress saved	per-browser (localStorage)	per-account (follows the student)
Prerequisite gating	open (all modules available)	enforced server-side
Instructor roster	no	yes
Best for	open access, demos, self-study	managed training cohorts

4. The data file (server mode)

- `academy_data.json` holds all accounts (passwords are **script-hashed**, never plain-text) and progress. It is created automatically on first run.
 - It is **gitignored and must never be committed** — it contains password hashes. The repo also ignores its rolling backup (`.bak`) and temp (`.tmp`) files.
 - The server writes atomically (temp file + rename) and keeps a rolling `.bak`, so an accidental crash or delete won't lose accounts. To relocate it, set the `ORBIT_DATA` environment variable.
 - **Back it up** by copying `academy_data.json` somewhere safe periodically.
-

5. How a student uses it

1. Open the hub (`index.html` / the site root). In server mode, register or log in.
2. Click a module. The **worksheet opens in its own window**; a **simulator** button opens the live tool. Arrange them side by side (worksheet left, simulator right works well).
3. In the worksheet: read the **objectives** and the one-page **tutorial**, then work the **numbered exercises** in the simulator, answering each check question (or clicking "mark done").
4. Finish with the **final check** (a short quiz over the whole module), review the **key points**, and follow any **resources** links.
5. Every worksheet header has buttons: **** Cheat sheet**** (keyboard/mouse controls, printable wallet card), **** Resources**** (all modules' links in one page), **** Print**** (print/save the worksheet as PDF), and ↗ **Simulator**.

Controls are the same in every simulator: **left-drag = pan**, **shift/right-drag = rotate**, **wheel = zoom**, **arrow keys = rotate**, **,/. = slow/speed time**, **space = pause**, **R = reset**.

6. Teaching notes, module by module

Each module is designed for roughly a **30-50 minute** session. The worksheet is self-paced; the instructor's role is to set context, watch for the common misconceptions below, and debrief.

Start every module with its intro deck. Each module ships an **8-slide PowerPoint intro deck** — `slides/module1.pptx` ... `slides/module8.pptx` — matching the lecture skele-

ton given below (hook → core beats with the anchor numbers → what students will do → the one thing to remember), with **speaker notes on every slide** so you can teach from it cold. The recommended rhythm for a session:

1. **Present the deck** (~10 minutes) to set the stage and pre-empt the module's misconception.
2. **Run the one live demo** called out in the module's lecture outline below, in the simulator, while the deck's demo slide is up.
3. **Release students to the worksheet** with the simulator open beside it, and circulate.
4. **Debrief** on the deck's final "one thing to remember" slide.

The decks are plain .pptx (PowerPoint, Keynote, LibreOffice, Google Slides all open them) and are safe to edit — they're teaching aids, not course data.

Module 1 — How Orbits Work. The one idea to land: an orbit is *falling sideways fast enough to keep missing the ground*. Common misconception: "there's no gravity in space." Have them do the speed-ladder exercise (too slow → crash, right speed → circle, faster → ellipse) before discussing. Payoff: geostationary = a 24-hour orbit that keeps pace with the ground.

Learning goals.

- Students can explain why a satellite stays up: it is falling continuously, but moving sideways fast enough that the ground curves away beneath it.
- Students can predict the path a given release speed produces — crash, circle, ellipse, or escape — and which end of an ellipse becomes perigee vs. apogee.
- Students can explain why an object runs fastest at perigee and slowest at apogee, and why higher orbits are slower overall (LEO ~7.6 km/s vs. GEO ~3.1 km/s).
- Students can state the difference between geosynchronous and geostationary, and explain why launch sites aim the way they do (east from Florida, south from Vandenberg).
- Students can place "space" at true scale: the ISS at ~420 km is only ~7% of an Earth radius up.

Intro lecture (10 minutes, before students open the worksheet).

- *Hook* — the ISS is only a DC-to-NYC drive away — straight up. "Space" is close; *staying* there is the hard part.
- *Beats* — (1) Newton's cannonball: gravity never turns off; throw something sideways fast enough and the ground curves away as fast as it falls — about 7.6 km/s at 600 km altitude. (2) Release speed decides the whole shape: too slow → crash, exactly right → circle, faster → ellipse, and at 10.7 km/s from 600 km (11.2 km/s from the surface) → gone forever. (3) Speed trades with height: fastest at perigee, slowest at apogee, and farther out means slower everywhere. (4) Push that to its payoff: at 35,786 km one lap takes one sidereal day (86,164 s) — the orbit keeps pace with the turning Earth.
- *Live demo* — set a release speed, then click **inject a fan (7 speeds, centered on your speed)** in tut1: one click shows the whole crash → circle → ellipse family side by side.
- *Misconception to pre-empt* — "there's no gravity in space." At ISS altitude gravity is still ~90% of its surface strength; astronauts float because they are falling *with*

their ship, not because gravity quit.

The deck for this module is `slides/module1.pptx` (9 slides matching this outline — including a **Driving the simulator** slide covering the mouse, the arrow keys, the `,/.` time keys and the control board, which pairs with worksheet Part A).

Module 2 — Angular Rates & Geosync. Central skill: angular rate vs. true speed, and why the GEO belt is prime, finite real estate (360 one-degree slots). The ground-telescope view (stars streak while a tracked satellite holds, and vice-versa) previews the observability module. Watch for confusion between *geosynchronous* (24-h period) and *geostationary* (24-h **and** equatorial).

Learning goals.

- Students can explain why apparent sky motion (angular rate) depends on both true speed and distance — and why a slow, distant GEO satellite can hang still while a faster LEO one streaks.
- Students can explain why the GEO belt is one finite ring with 360 one-degree slots, and why those slots are contested real estate.
- Students can distinguish geosynchronous from geostationary and predict the daily figure-8 ground track of a tilted 24-hour orbit.
- Students can predict what a 4-minute exposure shows on a fixed mount vs. a sidereal drive (stars streak vs. satellite streaks) and explain why no mount can freeze both.
- Students can explain why a geostationary satellite is motionless over the ground yet turns once a day against the stars — and why its solar panels must steer themselves.

Intro lecture (10 minutes, before students open the worksheet).

- *Hook* — point a camera at the night sky and it betrays itself: everything drifts at 15" per second — a full 15° every hour. Nothing overhead is still; the only question is what you choose to hold still.
- *Beats* — (1) Angular rate = speed ÷ distance: a LEO satellite crosses the sky in minutes, while a GEO satellite, ~85× farther out than the ISS, can appear frozen. (2) The magic ring: at 35,786 km altitude and 3.1 km/s, the period equals one sidereal day (86,164 s), so the satellite paces the turning Earth. (3) That ring is finite: 360 one-degree slots (real spacing ~2°) — prime, contested real estate. (4) Reference frames sneak in: "still" over the ground means turning once a day against the stars — which is why GEO solar panels rotate on their own joint.
- *Live demo* — the ground-telescope view in tut2: click ** TAKE A PICTURE (4 min exposure)** on the fixed mount (stars streak, the GEO sat is a point), then again on the sidereal drive (flipped).
- *Misconception to pre-empt* — "geosynchronous means it hovers." Only if it is also equatorial; tilt a 24-hour orbit and it traces a daily figure-8 over the ground.

The deck for this module is `slides/module2.pptx` (8 slides matching this outline).

Module 3 — Naming Orbits & TLEs. Goal: an orbit has a six-number "name tag" (the elements), and a TLE is just those numbers in a text format. The invariance idea is key — re-orienting an orbit (RAAN, argument of perigee) does **not** change its size, shape, or period. The Molniya "invent it by trial and error" exercise is a highlight; let students discover the ~12-hour period themselves. The **sun-synchronous preset** is the second designer-orbit payoff: the tool now draws the **Sun's direction** (advancing 0.9856°/day on the shared clock) and, with auto-move on, precesses the RAAN at the real **J2 nodal**

rate — at 700 km / i = 98.2° the node–Sun readout freezes (“*-synchronous”) while the ISS preset drifts $-5^\circ/\text{day}$. Same J2 that pins Molniya’s apogee: perturbations as tools, not nuisances.

Learning goals.

- Students can name the six element “sliders” — size, shape, tilt, swivel, twist, position — and say what each one controls.
- Students can explain why size alone sets the period, and why re-orienting an orbit (RAAN, argument of perigee) changes none of period, apogee, or perigee.
- Students can find the load-bearing fields of a TLE (epoch, inclination, eccentricity, mean motion) and connect each back to one of the sliders.
- Students can explain what makes a Molniya orbit work — 63.4° inclination, ~ 12 -hour period, $e \approx 0.74$ — and why its apogee loiters over high northern latitudes.
- Students can explain what makes a sun-synchronous orbit work — $\sim 98^\circ$ retrograde tilt so J2 precesses the plane at the Sun’s $+0.9856^\circ/\text{day}$ — and why imaging satellites want the same local solar time every pass.
- Students can explain why the orbit itself ignores Earth’s spin: it is set by gravity and the velocity in the fixed-star frame.

Intro lecture (10 minutes, before students open the worksheet).

- *Hook* — every object the Space Force tracks carries a name tag just six numbers long. Two lines of plain text, and you know where it will be next week.
- *Beats* — (1) Six sliders: two set the ellipse (size, shape), three aim it in space (tilt, swivel, twist), one places the satellite on it right now. (2) The invariance idea: swivel and twist re-aim the ellipse but leave size, shape, and period untouched — size alone is the clock.
(3) A TLE is just those numbers in fixed columns plus an epoch; mean motion (orbits per day) encodes the size and period. (4) Pose the design problem cold: GEO can only hover over the equator — so how does Moscow get all-day coverage of Siberia? Leave it hanging; the worksheet’s Molniya exercise (63.4° , ~ 12 h, $e \approx 0.74$) lets them invent the answer.
- *Live demo* — click **reset to a typical orbit** in tut3, then drag the swivel (RAAN) slider through a full turn while the class watches the period readout not move.
- *Misconception to pre-empt* — “turning the orbit changes the orbit.” Re-orientation changes what it flies over, not what it is — same size, shape, period, apogee, perigee.

The deck for this module is slides/module3.pptx (8 slides matching this outline).

Module 4 — Maneuvers & Perturbations. Two big ideas: **delta-v is finite currency** (the fuel gauge), and **tangential burns are efficient while radial burns are wasteful/paradoxical**. The GTO→GEO two-step transfer and the escape/re-entry scenarios are the payoff. Great counter-intuitive demos: the radial-burn “crash,” and the drag paradox (drag speeds a satellite up).

Learning goals.

- Students can explain delta-v as a finite currency and track each maneuver’s cost against a budget.
- Students can predict what a prograde or retrograde burn does (raises or lowers the *opposite* side of the orbit) and explain why radial burns barely change the orbit at all.

- Students can plan a two-step LEO→GEO transfer — raise apogee into a GTO, then a ~ 1.46 km/s prograde burn at apogee to circularize — and total the cost (~ 3.9 km/s).
- Students can explain the drag paradox: losing energy to drag drops a satellite to a lower orbit, where orbital speed is *higher*.
- Students can explain why perturbations (drag, J2, sunlight) make TLEs go stale, and why tracking must therefore be continuous.

Intro lecture (10 minutes, before students open the worksheet).

- *Hook* — in orbit, pushing “up” can drop you into the Earth, and air drag makes satellites speed up. This module is where everyday intuition goes to be recalibrated.
- *Beats* — (1) Delta-v is the currency: a satellite carries a fixed supply of speed-change, and when it’s gone the mission is over. (2) Burns pay off on the far side: a prograde burn *here* raises the orbit *there*; tangential burns change the orbit’s energy efficiently, radial burns do almost no work. (3) Every big move is two steps — raise apogee (that ellipse is a GTO), then circularize at apogee with a ~ 1.46 km/s prograde burn; LEO→GEO totals ~ 3.9 km/s. (4) Nothing coasts clean forever: drag, the equatorial bulge (J2), and sunlight all push orbits off their published elements — which is why TLEs expire.
- *Live demo* — ► **place satellite in circular orbit** in tut4, then hold ⌵ **radial out** and let the class watch the “crash”: fuel spent pushing up, and the satellite comes down.
- *Misconception to pre-empt* — “to go higher, thrust upward.” Orbits pay you sideways: to raise your altitude, speed up along your path — the height arrives half an orbit later.

The deck for this module is slides/module4.pptx (8 slides matching this outline).

Module 5 — xGEO / Cislunar Space & Reference Frames. The whole module is about **reference frames** — what you hold still changes everything. The **2×2 compare view** (all four frames at once, one clock) is the centerpiece; spend time there. The four frames are ECI, Earth-Moon rotating (synodic), Moon-centred inertial (MCI), and **ECL-EMBR** — the barycentric-rotating frame centered on the Earth-Moon barycenter, where *both* bodies freeze and the Lagrange points hold still (it sets up Module 6). The tool marks the barycenter with an orange cross at its **true position inside the Earth** — $\sim 4,670$ km from the center, 73% of the way to the surface, drawn through the globe (note: the sim keeps Earth pinned, so students should picture — not watch for — the real monthly Earth-wobble about that buried point) and shows the ecliptic vs. equator planes tilted by the 23.4° obliquity. Other key beats: the Hill sphere / L1 boundary (L1 is $\sim 85\%$ of the way to the Moon, *not* the midpoint), the definition of xGEO, and “leading the Moon” for a transfer. Note: the tool idealizes the Moon’s orbit as flat so the rotating frame reads cleanly; the real $\sim 5^\circ$ tilt is taught as the reason eclipses are occasional.

Learning goals.

- Students can define xGEO — beyond the GEO belt ($\sim 36,000$ km) but inside Earth’s Hill sphere (~ 1.5 million km) — and place the Moon ($384,400$ km ≈ 60 Earth radii, 27.3-day period) inside it.
- Students can explain why the L1 balance point sits $\sim 85\%$ of the way to the Moon ($\sim 326,000$ km), not at the midpoint, and predict which body an object released at rest on the line falls toward.
- Students can pick the right reference frame for a given question — ECI, Earth-Moon

rotating (synodic), MCI, or ECL-EMBR — and say what each one holds still.

- Students can locate the Earth-Moon barycenter (4,670 km from Earth's center — inside the planet) and say why the ECL-EMBR frame is centered there.
- Students can explain why a lunar transfer must lead the Moon: you burn toward where it will be days from now, not where it is.

Intro lecture (10 minutes, before students open the worksheet).

- *Hook* — the Moon doesn't orbit the Earth. Both orbit a shared balance point 4,670 km from Earth's center — a point buried ~1,700 km beneath our feet.
- *Beats* — (1) The scale jump: the Moon is 384,400 km out — about 60 Earth radii, a 27.3-day lap; GEO, our "high ground" so far, is barely a tenth of the way there. (2) The tug-of-war: whose satellite are you? The Moon's territory is its Hill sphere, and the along-the-line handoff point (L1) sits ~85% of the way out, because the Moon is the lightweight. (3) Frames are the skill: the same month of motion looks like a tangle or a still-life depending on what you hold fixed — four frames, one clock. (4) Getting there means leading the Moon like a duck hunter: you aim at where it will be when you arrive, not where it is when you burn.
- *Live demo* — ■ **Compare all 4 frames at once (2×2)** in tut5: one satellite, one clock, four pictures; let it run a full month while the class watches which panels stay simple.
- *Misconception to pre-empt* — "the balance point is halfway to the Moon." Earth outweighs the Moon ~81:1, so the handoff sits ~85% of the way out — most of cislunar space belongs to Earth.

The deck for this module is slides/module5.pptx (8 slides matching this outline).

Module 6 — Lagrange Points & Complex Orbits. The richest, most conceptually demanding module. Core beats: the five Lagrange points as **spots that share the Moon's 27.3-day period** (drawn as ▲ markers, distinct from bodies); the **1-D force-balance** view (L1 the Moon opposes Earth → gentler pull; L2 they add → stronger pull; "balance" never means zero net pull); **L4/L5 stable, L1/L2/L3 unstable** (golf-ball-on-a-basketball); the **near-rectilinear halo orbit (NRHO)** that CAPSTONE/ Gateway fly; **TESS** in 2:1 resonance; the **six fan-release experiments** (live RK4, true restricted three-body field): ** lunar scatter** (slingshot family; the deepest skim flung past Earth's Hill sphere), □ **L1 knife-edge** (seven objects at L1, ≤8 m/s kicks decide moonward vs earthward vs weeks-long hover — unstable equilibrium made visceral), ** L4 tadpoles** (the same kicks and bigger just librate; a 50 m/s outlier finds the pocket's rim — the Trojan-asteroid story), ** DRO vs prograde** (Earth's tide strips distant prograde lunar orbits while every distant retrograde orbit persists — why Artemis I parked Orion in a DRO), ∞ **free return** (a ~6 m/s departure spread decides re-entry vs stranding vs lunar impact — the Apollo 13 fail-safe), and ** minimoons** (identical unbound arrivals differing only in arrival DAY; lunar flybys ballistically capture some as temporary moons of Earth for weeks to over a year — the real 2006 RH120 / 2020 CD3 phenomenon); the **gravity landscape** overlay — the co-rotating effective-potential surface (Earth/Moon funnels, L1/L2/L3 saddle passes, L4/L5 hilltops) that makes "ball in a bowl vs ball on a saddle" literal; and the capstone **libration "zoo"** — the whole family of L1 orbits from two amplitude knobs and a frequency ratio. This module rewards an instructor who understands the physics deeply; **\$11 below is a dedicated deep-dive on the L1 orbits** because the questions this module provokes are genuinely subtle. Hard language rules the module obeys: never say "centrifugal"/"centripetal" — motion is explained with real gravity + sideways motion.

Learning goals.

- Students can define a Lagrange point as a spot where Earth + Moon together give an object the Moon's own 27.3-day period — and explain why “balance” never means zero net pull.
- Students can explain the 1-D force balance along the Earth-Moon line: at L1 the Moon opposes Earth (a gentler pull suits the closer-in spot); at L2 the pulls add (a stronger pull suits the farther-out spot).
- Students can state which points are stable (L4/L5) and which are not (L1/L2/L3), and why missions at the unstable ones fly halo orbits — including Gateway's NRHO — with regular stationkeeping.
- Students can explain orbital resonance through TESS's 2:1 lock with the Moon ($P = 13.66$ d): the Moon's repeated tugs are arranged to cancel over time.
- Students can describe chaos operationally: near-identical starts diverge within days, so cislunar predictions go stale fast — and can cite the knife-edge (≤ 8 m/s decides the fate) and free-return (~ 6 m/s decides re-entry vs stranding) releases as concrete evidence.
- Students can name the stable real estate of cislunar space — L4/L5 tadpoles and distant retrograde orbits (DROs) — and connect each to a real anchor (Jupiter's Trojans; Artemis I's Orion parking).
- Students can explain a free-return trajectory and why Apollo flew one (no burn needed at the Moon to come home — what saved Apollo 13).
- Students can explain ballistic capture: a lunar flyby can hand an unbound arrival enough energy to the Moon to leave it temporarily bound — Earth's real minimoons (2006 RH120, 2020 CD3) — and can read the gravity-landscape surface (funnels, saddles, hilltops) as the map of all of the above.

Intro lecture (10 minutes, before students open the worksheet).

- *Hook* — there are five parking spots in the Earth-Moon system where you can hold formation with the Moon forever. Three of them sit on a knife's edge — and that's exactly where we want to put things.
- *Beats* — (1) A Lagrange point is not zero gravity: it's where the leftover pull is *exactly* what a 27.3-day loop at that distance requires. (2) The 1-D balance: at L1 ($\sim 85\%$ of the way out) the Moon cancels part of Earth's pull; at L2 ($\sim 117\%$ out, $\sim 449,000$ km) it adds to it — and both spots still match the lunar period. (3) Stability is the golf ball on the basketball: L1/L2/L3 need constant nudging, so spacecraft fly looping halo orbits around them; L4/L5 are the bowls, where dust can collect. (4) Off the special solutions lies real chaos — tiny release differences diverge within days, which is why cislunar custody (Module 7's subject) is genuinely hard.
- *Live demo* — the **** Fan release**** scenario in tut6. Best 3-minute pairing: turn on the **gravity landscape** in the synodic frame (saddles and hilltops visible), then run **L1 knife-edge** (objects hover at L1 for days, then ≤ 8 m/s kicks send them to opposite fates), then **** L4 tadpoles**** (bigger kicks, and everything just loops in place) — stable vs unstable, with the terrain that explains it in view. If time allows, **** lunar scatter**** viewed in the Moon-centered inertial frame is the crowd-pleaser (clean slingshot hairpins, one object flung past Earth's Hill sphere), and **** minimoons**** is the closer — Earth catching temporary moons live.
- *Misconception to pre-empt* — “a Lagrange point is where gravity cancels to zero.” At L2 the two pulls *add* — the net pull there is stronger, not absent; balance means

matching the Moon's period, not floating force-free.

The deck for this module is `slides/module6.pptx` (8 slides matching this outline).

Module 7 — Observability. The “how we actually see satellites” module (we can't see *space* — only the objects in it) — the practical payoff of the whole course, and the bridge to space domain awareness. Core beats: **active radar** (received power falls as **range⁴** two-way — a GEO target returns $(42,164/800)^4 \approx 7.7$ **million $\times$** weaker than an 800-km LEO one, the numbers the radar-sim scene uses, and xGEO is hopeless for most radars); the **“headlights” analogy** for optical (you only see sunlit objects — you're driving at night seeing bicycles only when a passing truck's headlights, the Sun, catch them); **thermal-IR** self-emission (warm objects glow even in shadow); **cooperative vs. uncooperative** tracking; **custody and cadence** (chaotic cislunar orbits go stale fast, so you must re-observe often); **maneuver detection**; the crucial distinction between **orbitology** (where is it) vs. **characterization** (what is it) vs. **inferring intent** (why); **proximity operations / neighborhood watch**; how radar measures **range and range-rate** with crude angles; how optical gives precise **angles (RA/DEC)** but no range directly; how **parallax** across successive images (or two sites) constrains range (the finger-in-front-of-alternating-eyes demo); and **pointed custody vs. all-sky survey** tradeoffs.

(v5.5: this module was revised end-to-end from an external non-astronomer review — the worksheet now carries a magnitude reference table with the $m / M / \text{“mag”}$ notation warning, defines every symbol at first use, derives the $\sqrt{N}$ integration gain, and warns that Tag-&-fit needs time-spread tags before DETERMINED. Exercises without a sim scene are labeled “Think it through”. If a student asks why Hubble can't image a GEO satellite's panels: its $\sim 0.05''$ diffraction limit spans ~ 9 m at GEO range — proximity operations is the honest answer, and the worksheet now says so.)

Learning goals.

- Students can explain why a radar echo falls as range^4 and reproduce the headline ratio: a GEO target returns $(42,164/800)^4 \approx 7.7$ million $\times$ less power than an 800-km LEO one.
- Students can predict when optical tracking works (sunlit target, dark sky), when it fails (Earth's shadow), and when thermal-IR still sees the object.
- Students can say what each sensor measures — radar: range and range-rate with crude angles; optical: precise angles (RA/DEC) with no range — and explain how parallax recovers the range.
- Students can explain custody and cadence — re-observe before the prediction goes stale — and how comparing an observation to the prediction detects maneuvers.
- Students can place a question on the ladder: orbitology (where is it) $\rightarrow$ characterization (what is it) $\rightarrow$ intent (why).

Intro lecture (10 minutes, before students open the worksheet).

- *Hook* — the same radar pulse that pings an 800-km satellite loud and clear comes back from GEO 7.7 million times weaker. So how do we know where *anything* out there is?
- *Beats* — (1) Three ways to sense a space object: bounce radar off it, catch the sunlight it reflects, or catch the heat it emits. (2) Radar is active — it transmits its own pulses, so it needs no sunlight — but pays range^4 — it owns LEO, strains at GEO, and is hopeless for most of xGEO. (3) Optical is driving at night: you see the bicycle

only when a passing truck's headlights — the Sun — catch it; in Earth's shadow it goes blind, and thermal-IR takes over. (4) No sensor tells you everything: radar gives range, optical gives angles, parallax stitches range back in — and custody means re-observing on a cadence before a (possibly chaotic) orbit goes stale, which is also how maneuvers are caught.

- *Live demo* — the **** Radar sim**** scene in tut7: **FIRE PULSE** at the LEO target, then at the GEO one, and let the class watch the echo vanish into the noise (then **START AVERAGING**).
- *Misconception to pre-empt* — “we track everything all the time, like air-traffic control.” Sensors see a tiny patch of sky; the catalog is stitched from scheduled glimpses, and objects can go stale — or maneuver — between looks.

The deck for this module is slides/module7.pptx (8 slides matching this outline).

Module 8 — Lunar Transfers & Artemis (the capstone). The student stops studying orbits and **flies** one, from an Artemis-style cockpit. Two views: a **PLAN** mode (a flight computer where you compute each burn's Δv from vis-viva and iterate to the target, with a live trajectory predictor and a “run sim” playback) and a **FLY** mode (the cockpit — window with real Earth/Moon at correct angular size, embedded MFD screens, hold-to-thrust controls, burn **countdowns**, and a BURN-NOW cue whose numbers are **live guidance** recomputed from the current orbit). Two missions, flown in order: **go to GEO** (a gentle two-burn Hohmann warm-up, familiar from Modules 2-3), and — **unlocked by a successful GEO run — go to the Moon**, a genuine **three-burn** profile (aim just *past* it, lead it $\sim 120^\circ$; a retrograde **LOI brake** at closest approach; then a **cued circularization** in lunar orbit; the planner grades lunar plans with the full three-body sim, since no clean vis-viva check exists for capture). Thrust acts in the **local flight frame** — Earth-relative normally, Moon-relative inside its sphere of influence — so REVERSE genuinely brakes a lunar orbit. Every mission starts with one full parking lap before the first cue, and an **** AUTO FLY**** autopilot can fly the whole order book through the same controls as the pilot — a good demonstrator for students who stall (watch it, then beat it by hand). Victory means *holding* the target orbit for a full revolution, not just touching it. Everything from Modules 1-6 gets used in anger.

Learning goals.

- Students can build a Δv logbook — each burn's cost and a running total — and judge before flying whether a mission fits the tank.
- Students can plan the LEO→GEO transfer: raise apogee ($\sim 2,400$ m/s), then circularize ($\sim 1,460$ m/s), $\approx 3,860$ m/s total against the 6,500 m/s budget — and explain why circularizing costs so much (you arrive at apogee moving slowly and must speed up to GEO's 3.1 km/s).
- Students can plan the lunar leg: lead the Moon $\sim 120^\circ$, aim apogee just *past* it, TLI $\sim 3,090$ m/s, a ~ 4 -day coast, then LOI — a ~ 900 m/s retrograde brake at closest approach — inside a 9,500 m/s budget.
- Students can fly the plan on cue: hold thrust to the logged Δv at BURN NOW, stay on the ILS-style track and speed indicators, and hold the target orbit for a full revolution.

Intro lecture (10 minutes, before students open the worksheet).

- *Hook* — today you stop studying orbits and fly one. Your fuel gauge is the exam.
- *Beats* — (1) **What Δv is:** a burn changes your velocity, and propellant buys that

change at an exponential rate, so the tank (6,500 m/s for GEO, 9,500 for the Moon) is the mission's real limit.

- (2) **Send them to TRAIN first** — free flight, unlimited fuel, nothing scored — because the next beat has to be *felt*, not told. (3) The **counter-intuitive rule**: to catch a satellite ahead of you, burn REVERSE. A lower orbit has a shorter period, so you come round sooner and creep up on it ($\delta T/T = 1.5 \cdot \delta a/a \rightarrow \sim 5^\circ/\text{day}$ for a 400 km dip; 1° of the GEO belt is 736 km). Your speed at the burn point actually *falls* — it is the period that wins the race. (4) The warm-up mission: service a GEO satellite — $2,400 + 1,460 \approx 3,860$ m/s against a 6,500 m/s tank — and the plan is only part of the job: you arrive $\sim 8^\circ$ **behind** the target and phase in, which is what the balance of the tank pays for. (5) The graduation flight (unlocked only by a clean GEO run): the Moon *moves* while you coast ~ 4 days, so lead it $\sim 120^\circ$ and aim $\sim 395,000$ km — just **past** it. TLI $\sim 3,090$ m/s, and it is a knife edge: ± 1.5 m/s flips a capture into an impact or a miss. (6) **Three** burns, not two: at closest approach you turn around and brake (~ 600 m/s to the local circular speed — *less* than the 900 in the logbook), and because capture leaves an **ellipse**, a third burn rounds it out **at an apsis**, the only place where one along-track burn can. (7) Say the scoring out loud: you must **hold** the target box a full revolution, not just touch it.
- *Live demo* — in PLAN mode in tut8, pick **** Service a GEO satellite, click ☐ solve it for me, then ► GO — run sim: the predictor flies the whole two-burn transfer end to end. If time allows, fly the GEO mission once with AUTO FLY** on** and let the class read the Δv and the grade off the debrief — it sets the standard they are trying to match by hand.
 - *Misconceptions to pre-empt* — (a) “to get captured, speed up toward the Moon.” Arrive without braking and the Moon slingshots you right past; capture is the retro-grade burn at closest approach. (b) “to catch something ahead of me, thrust toward it.” That raises your orbit, lengthens your period, and it pulls further away.

The deck for this module is slides/module8.pptx — **12 slides** matching these beats: title · why you care · **Δv is the currency** · Mission 1 (GEO servicing) · **To catch up, drop LOWER** · Mission 2 (lead the Moon) · Lunar Orbit Insertion · **Circularize only at an apsis** · the cockpit (TRAIN / PLAN / FLY + the rendezvous director) · **How you're scored** · what you'll do · the one thing to remember.

7. Instructor dashboard (server mode)

Log in with the instructor account (the first one registered). The hub shows a **roster**: each student, which modules they've completed, and their pass status. Prerequisite gating means a student can't skip ahead — the server enforces it regardless of what the browser does.

To reset a cohort, stop the server and archive/remove academy_data.json (keep a backup first); a fresh file is created on next start, and the first new registration becomes the instructor again.

7.1 The worksheet editor — change the course content without touching code

The instructor dashboard has an `✎ Edit worksheets` button (it opens `editor.html`; instructor accounts only — students are refused by the server, not just the page). It lets you rewrite any worksheet — objectives, tutorial text, every exercise’s teach/predict/do/observe/think blocks, the quiz questions, options, correct answers and per-option feedback, and the final-check questions — from a form, with no code editing.

What you need to know before using it (all verified against the running server):

- **What it edits:** the *published* workbook set in `workbooks/active/` — the files the server actually serves to students. Saves take effect on each student’s **next worksheet load** (the pages are served no-cache, so a browser refresh picks the edit up).
- **It’s hard to break things.** Every save is first executed in a sandbox and must produce a valid worksheet — a save that would white-screen a worksheet is **rejected** with an error and the old version stays live. Writes are atomic, and every successful save first snapshots the outgoing version as a timestamped `worksheet<N>.data.<date>.bak` beside it, so any edit can be undone by renaming the `.bak` back over the `.data.js` file.
- **One caveat — standalone mode:** the zero-install/no-server mode serves its *own* copies of the worksheets (in `public/`). Editor saves do **not** touch those. If you use standalone mode and want your edits there too, copy the files across after editing: `cp workbooks/active/worksheet*.data.js public/` (and note the copies lose the source files’ code comments — the content itself round-trips exactly).
- **Versioned sets:** sibling folders under `workbooks/` (e.g. `workbooks/2026-spring/`) hold inactive versions; “publishing” a set is just renaming folders so yours is called `active`. Works the same on Windows and macOS.

A sensible workflow: keep the class on `active`, clone it to a dated folder before a big revision (`cp -r workbooks/active workbooks/2026-fall-draft`), edit live, and if a revision goes sideways, swap the folder names back.

8. Updating & testing

- Pull the latest files (ZIP or `git pull`).
- If you run the server, sanity-check it after updating:

```
cd academy
```

```
bash test/run.sh          # functional + security + DoS checks, on an isolated temp data
```

The tests never touch your real `academy_data.json`.

- See `docs/CHANGELOG.md` for what changed, and `docs/SECURITY.md` for the per-release audit log.

9. Troubleshooting

- **“Please sign in first.”** — You’re in server mode but not logged in. Register/log in at the hub. (If you *want* no-login use, host the static files without the server — §3 Option 1.)
 - **The worksheet window didn’t open.** — The browser blocked the pop-up. Allow pop-ups for the site, then click “Re-open the worksheet.”
 - **3-D view is blank, or a banner says “3-D library not loaded”.** — `public/vendor/three.min.js` is missing from the course folder; it ships in the repository, so this means an incomplete copy or extraction, **not** a missing internet connection (the course never needs one). Run `bash tools/fetch-vendor.sh --check`, or restore with `git checkout -- public/vendor`. A missing *texture* is harmless by comparison — the sim falls back to the schematic map that also ships in the repo.
 - **Progress didn’t save across devices.** — That’s standalone mode (per-browser storage). Use the server (Option 2) for progress that follows the student.
 - **Lost accounts.** — Restore `academy_data.json` from its `.bak` (same folder) or your backup.
-

11. Deep dive — the complexities of L1 (halo) orbits

Module 6’s libration “zoo” and NRHO scenario provoke sharp questions. This section arms you to field them. It is background for the instructor, pitched above the student level.

1. These are not Keplerian orbits. A satellite in LEO or around the Moon traces an **ellipse** about a single dominant body, fully named by six orbital elements (a TLE). A **halo orbit around a Lagrange point has no single central body** — it circles an *empty point in space* where Earth’s and the Moon’s pulls, together with the motion needed to keep pace with the rotating Earth-Moon line, balance out. There is **no focus, no fixed ellipse, no TLE**. It is a periodic solution of the *restricted three-body problem*. When a student says “but it’s not centered on anything!” — exactly. That’s the whole point, and it’s the single most mind-expanding idea in the course.

2. One parameter, not six. Astonishingly, the entire family of halos around a given Lagrange point is labeled by **one number** — the **Jacobi constant**, the conserved energy-like quantity of the rotating frame (higher amplitude $\leftrightarrow$ lower Jacobi constant). Fix it and the size, the out-of-plane height, and the period are all determined. Contrast the six-plus elements of an ordinary orbit. In the zoo, the amplitude sliders + frequency-ratio buttons expose this: at a **1:1** frequency lock the in-plane and out-of-plane amplitudes are tied together by a constraint (one free parameter $\rightarrow$ a true halo); at other ratios they’re independent (a two-parameter **Lissajous/quasi-periodic** family that never closes). A whole-number ratio closes into a figure; an irrational ratio fills a 3-D band forever.

3. Why the near-degeneracy matters. Near a collinear point the motion splits into an unstable saddle $\times$ an in-plane oscillation (ω_p) $\times$ an out-of-plane oscillation (ω_v). For Earth-Moon L1 these are **$\omega_p \approx 2.37$, $\omega_v \approx 2.30$** (units of the monthly rate) — very close but unequal. A generic bounded orbit is therefore a **Lissajous** that never closes. A **halo** is the special amplitude at which nonlinear terms drag ω_p and ω_v into an exact

1:1 lock so the path closes into a single loop. Because the two frequencies start so close, that lock happens at modest amplitude — which is why real Sun-Earth L1/L2 halos (SOHO, Gaia, JWST) are rounded, moderate loops, not wild shapes.

4. The NRHO is deliberately lopsided — and that’s correct. A near-rectilinear halo (CAPSTONE, Gateway) skims a few thousand km over one lunar pole and swings ~70,000 km over the other. Students (and instructors) balk: the gravity field is mirror-symmetric top-to-bottom, so shouldn’t the orbit be? **No.** A symmetric field yields a mirror-image *pair* of solutions — a **northern** and a **southern** halo — and each individual orbit picks a side (like a ball settling into one well of a symmetric double-well). Gateway flies a southern NRHO; its northern twin is equally valid. The orbit *is* still symmetric, just about the plane through the Earth-Moon line and the poles, not the orbital plane. **Second gem:** because it’s a very eccentric loop, Kepler’s second law applies locally — the craft **whips through the close pole pass in hours but loiters for days at the far end**, so a “lunar” orbit spends ~99% of its time *far* from the Moon.

5. What supplies the torque? (The subtle one.) In the rotating frame the halo holds a fixed orientation, so in the inertial sky its orientation — and its angular-momentum vector — **sweeps around once a month.** A changing angular momentum requires a **real torque.** Where from? **The Moon’s off-axis gravity.** Earth’s pull is central about Earth (zero torque about Earth), but the Moon sits off to the side, so its tug does not point through Earth and exerts a genuine torque that swings the orbit around to keep pace with the Earth-Moon line. This is *not* a rotating-frame artifact — it is honest Newtonian gravity, and it vindicates the course’s “banish fictitious forces” rule. (Numerically the Moon’s torque about Earth on a representative halo point is ~0.2 in km²/s² per unit mass — nonzero, exactly as required.)

6. Honesty about the tool. The zoo uses the **linearized** (Richardson) equations plus a lookup-table nudge for how the loop’s center migrates toward the Moon at high amplitude. The shapes and rhythms are faithful; it is **not** a full nonlinear integration, and it is labeled as such in the UI. If a student asks whether they could fly one, the answer is yes — but station-keeping is required because L1/L2/L3 are unstable (the halo is a controlled dance around an unstable point).

12. Credits & license

MIT licensed. Built for JASON / US Space Force training; companion to the CISLUNAR PATROL game and the xGEO simulator. Three.js is © its authors (MIT), vendored into public/vendor/ and hash-verified — nothing is loaded from a CDN.